

Summer Term 2025
Computer Animation - Assignment Sheet 1
 Due Date May 8, 2025

Assignment 1 (*Rigid Body Transformations, 4 Points*)

A linear mapping $g : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ is a rigid body transformation, if and only if

$$\|g(\mathbf{p}_1) - g(\mathbf{p}_2)\| = \|\mathbf{p}_1 - \mathbf{p}_2\|, \quad (1)$$

holds for all points $\mathbf{p}_1, \mathbf{p}_2 \in \mathbb{R}^3$ and

$$g_*(\mathbf{v}_1 \times \mathbf{v}_2) = g_*(\mathbf{v}_1) \times g_*(\mathbf{v}_2), \quad (2)$$

holds for all $\mathbf{v}_1, \mathbf{v}_2 \in \mathbb{R}^3$. In this definition g induces an action on vectors in a natural way by $g_*(\mathbf{v}_i) = g(\mathbf{p}) - g(\mathbf{q})$ for vectors $\mathbf{v}_i = \mathbf{p} - \mathbf{q}$ connecting points $\mathbf{p}$ and $\mathbf{q}$ in $\mathbb{R}^3$.

1. Describe what each part of this formal definition means, geometrically.
2. Create a technical sketch of a **rotation** in $\mathbb{R}^3$: You may choose an everyday object of your liking or you may sketch vectors in $\mathbb{R}^3$. Sketch your coordinate system and indicate which axis is which. Give the corresponding rotation matrix in $\mathbb{R}^{3 \times 3}$ and check whether it satisfies the definition of a rigid body transformation.
3. In the same fashion, sketch a **translation** with **scaling** for an object of your choice. Give a description by a mathematical term (including parameters) for this mapping and argue whether this is a case of a rigid body transformation.
4. Sketch a **sheared** object of your choice, give the corresponding transformation matrix. Check whether this is a rigid body transformation.

Assignment 2 (*Special Orthogonal Group, 2 + 1 + 1 = 4 Points*)

The special orthogonal group of dimension n is defined by

$$\mathbf{SO}(n) := \{\mathbf{R} \in \mathbb{R}^{n \times n} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{Id}_n, \det(\mathbf{R}) = 1\},$$

in which $\mathbf{Id}_n$ denotes the $(n \times n)$ -identity matrix.

1. Prove that $G := (\mathbf{SO}(n), \cdot)$ is a group, in which “ $\cdot$ ” denotes the usual matrix multiplication (i.e. show closure, existence of identity and inverse elements, and associativity).
 For which $n \in \mathbb{N}$ is G abelian?

An element $\mathbf{R} = [\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_n] \in \mathbf{SO}(n)$ is called an n -dimensional rotation matrix. Prove the following statements.

2. From $\mathbf{R}^\top \mathbf{R} = \mathbf{Id}_n$ follows $\forall (i, j) \in \{1, 2, \dots, n\}^2 : \langle \mathbf{r}_i | \mathbf{r}_j \rangle = \delta_{ij}$.
3. Consider the set defined by

$$\mathbf{S} := \{\mathbf{R} \in \mathbb{R}^{n \times n} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{Id}_n, \det(\mathbf{R}) = -1\}.$$

Are the elements of this set also rotations? Does $\mathbf{S}$ form a group?

Assignment 3 (*Rotation Matrices, Euler Angles, and Quaternions, $2 + 2 + 2 = 6$ Points*)

Implement the following functions in Python using NumPy for the representation and conversion of rotation matrices, Euler angles, and quaternions. The Euler-angle convention must be passed explicitly to all functions that convert from or to Euler angles (for example as a string such as XYZ or ZYX).

1. Implement the following functions for rotation matrices:

- Conversion quaternion $\rightarrow$ rotation matrix.
- Conversion Euler angles $\rightarrow$ rotation matrix.

2. For quaternions please implement the following functions:

- Conjugation,
- Multiplication,
- Norm,
- Conversion rotation matrix $\rightarrow$ quaternion.
- Conversion Euler angles $\rightarrow$ quaternion.

3. For Euler angles please implement the functions:

- Conversion quaternion $\rightarrow$ Euler angles.
- Conversion rotation matrix $\rightarrow$ Euler angles.

Thereby, a quaternion shall be represented as a 4×1 -matrix, Euler angles as 3×1 -matrix.

Assignment 4 (*Quaternion Functions, 5 Points*)

Extend the functions for conjugation, multiplication and norm of quaternions from the previous assignment, so that a $4 \times n$ -matrix, consisting of n -rotations, shall be considered, instead of a single quaternion. For the multiplication it should be allowed to multiply two matrices of same size, or one matrix with a unique quaternion.

Keep this in mind: instead of running through a loop n times save time by making use of element-wise operations when implementing multiplication functions!

```
import numpy as np
```

```
a = np.array([[1, 1, 1],  
              [1, 2, 3],  
              [1, 3, 6]])
```

```
a @ a    # matrix multiplication  
a * a    # element-wise multiplication
```

Notes

- The assignment is due **Sunday, 3 May, at 23:59**.
- Python and NumPy will be used for the programming assignments. You may use your own Python installation or the computers in the B-IT Pool U1.108
- Please choose a timeslot for the exercise presentation as soon as possible or tell us if you do not have time during the proposed timeslots so we can plan accordingly.
- You have to upload your solutions to eCampus (single file upload, ZIP file with your script(s), name the ZIP file according to your last names, upload all necessary files so that it can run 'out of the box') until the due date. Points are given after demonstrating and discussing the solutions in the tutorials with your tutor.
- In case you have any questions about the assignments, please contact Melissa Steininger (s29mstei@uni-bonn.de)